

通常 Tello + Raspberry Pi ゲートウェイ構成 追加実装手順書

2026 年 6 月 9 日

1 目的

本手順書では、通常モデルの DJI Tello を複数台用いる協調取り囲み実験のために追加したソースコード群の導入手順をまとめる。通常 Tello は、各機体が Wi-Fi アクセスポイントとして動作し、Tello 側 IP アドレスが原則として各機体で同じ 192.168.10.1 になる。そのため、1 台の PC から複数の通常 Tello へ直接接続して制御する構成は取りにくい。

そこで、Tello の台数分の Raspberry Pi を用意し、各 Raspberry Pi を 1 台の Tello 専用ゲートウェイとして使う。デスクトップ PC は有線 LAN で各 Raspberry Pi と通信し、各 Raspberry Pi は Wi-Fi で対応する Tello に接続する。PC 側は外部計測、安全シールド、目標スロット計算、方策またはベースライン制御を担当し、Raspberry Pi 側は Tello SDK コマンド送信を担当する。

2 追加ファイル構成

追加したファイル構成は次の通りである。

```
tello_pi_gateway_patch/ |——
python/ |
  |—— run_real_baseline.py |
  |—— tello_rl/ |
    |—— real/ |
      |—— __init__.py |
      |—— pi_client.py |
      |—— real_tracker.py |
      |—— real_safety_shield.py |
      |—— real_tello_env.py |——
raspberry_pi/ |
  |—— tello_gateway.py |——
configs/ |
  |—— pc_real_config.example.json |
  |—— tracker_udp_schema.example.json |
  |—— pi_gateway_commands.txt |——
README_source_files.md
```

各ファイルの役割を表 1 に示す.

表 1: 追加ファイルの役割

ファイル	役割
<code>run_real_baseline.py</code>	実機構成で PD ベースライン制御を実行する PC 側エントリ. 学習済み方策を使う前の通信・外部計測・安全シールド確認に用いる.
<code>pi_client.py</code>	PC から各 Raspberry Pi の <code>tello_gateway.py</code> へ TCP/JSON Lines で指令を送るクライアント.
<code>real_tracker.py</code>	外部計測系から UDP JSON で送られてくる Tello と対象物の状態を受け取る.
<code>real_safety_shield.py</code>	実機用の安全シールド. 機体間距離, 壁距離, 高度範囲, 入力上限, 計測欠落を監視する.
<code>real_tello_env.py</code>	既存の <code>MockEnv/UnityEnv</code> と同様の API を持つ実機環境ラップ. 将来的に学習済み方策を実機へ接続するための入口.
<code>tello_gateway.py</code>	Raspberry Pi 上で実行する Tello SDK ゲートウェイ. PC から受け取った正規化行動を <code>rc a b c d</code> へ変換して Tello に送る.
<code>pc_real_config.example.json</code>	PC 側の実機構成設定例. Pi の IP, 飛行領域, 安全距離, 制御ゲインなどを定義する.
<code>tracker_udp_schema.example.json</code>	外部計測系が PC へ送る UDP JSON の形式例.
<code>pi_gateway_commands.txt</code>	Pi ゲートウェイに送れるメッセージ形式のメモ.

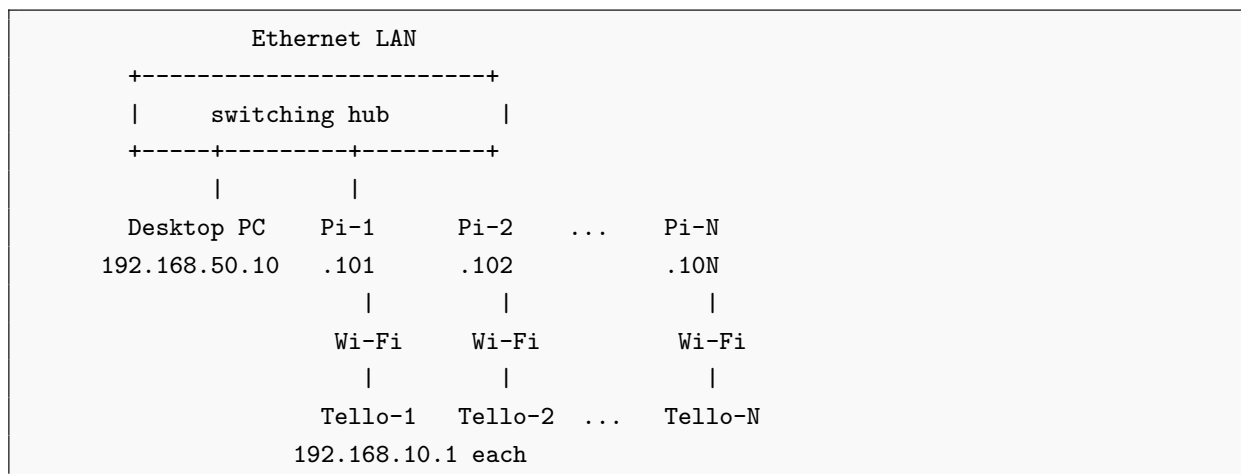
3 前提条件

本実装は, 以下の前提で動作する.

1. デスクトップ PC と全 Raspberry Pi は, 同じ有線 LAN に接続されている.
2. 各 Raspberry Pi は, `wlan0` で対応する 1 台の Tello の Wi-Fi に接続する.
3. PC から Tello の `192.168.10.1` へは直接アクセスしない. PC は各 Raspberry Pi の有線 LAN 側 IP アドレスだけを扱う.
4. 外部計測系が存在し, Tello の水平位置, 高度, `yaw`, 対象物位置を PC に送信できる.
5. 初期実験では, 学習済み方策ではなく PD ベースライン制御を用いる.
6. 実機飛行時は, 安全ネット, 十分な飛行領域, 緊急停止手段, 予備バッテリーを用意する.

4 ネットワーク構成

推奨するネットワーク構成を以下に示す.



有線 LAN 側 IP アドレスの例を表 2 に示す.

表 2 有線 LAN 側 IP アドレス設定例

機器	インターフェース	IP アドレス
デスクトップ PC	Ethernet	192.168.50.10/24
Raspberry Pi 1	eth0	192.168.50.101/24
Raspberry Pi 2	eth0	192.168.50.102/24
Raspberry Pi 3	eth0	192.168.50.103/24

Raspberry Pi は IP ルータとしては使わない. したがって, IP forwarding や bridge 設定は不要である. Raspberry Pi は PC と Tello の間でアプリケーション層の変換を行うだけである.

5 Raspberry Pi 側の準備

5.1 ファイル配置

各 Raspberry Pi に, 次のファイルをコピーする.

```
raspberrypi/tello_gateway.py
```

例えば PC から Raspberry Pi へコピーする場合は次のようにする.

```
scp raspberrypi/tello_gateway.py pi@192.168.50.101:~/tello_gateway.py
```

Pi が複数台ある場合は, 各 Pi に同じ `tello_gateway.py` をコピーする. Pi ごとの差分は, 起動時の引数または接続先 Tello Wi-Fi の SSID で管理する.

5.2 有線 LAN 側 IP の固定

Raspberry Pi OS で NetworkManager を用いる場合、Pi-1 では次のように有線 LAN 側 IP を固定する。

```
sudo nmcli con mod "Wired connection 1" ipv4.addresses 192.168.50.101/24
sudo nmcli con mod "Wired connection 1" ipv4.method manual
sudo nmcli con up "Wired connection 1"
```

Pi-2 では 192.168.50.102/24, Pi-3 では 192.168.50.103/24 のように割り当てる。

5.3 Tello Wi-Fi への接続

各 Raspberry Pi の wlan0 を, 対応する Tello の Wi-Fi に接続する。

```
sudo nmcli dev wifi rescan
nmcli dev wifi list
sudo nmcli dev wifi connect TELLO-XXXXXX ifname wlan0
```

接続後, Pi から Tello に到達できるか確認する。

```
ping 192.168.10.1
```

5.4 Pi ゲートウェイの起動

各 Raspberry Pi で `tello_gateway.py` を起動する。初期確認では, `--auto-command` を付けて起動時に Tello SDK mode へ入るようにする。

```
python3 ~/tello_gateway.py \
  --listen-host 0.0.0.0 \
  --listen-port 10000 \
  --auto-command \
  --rc-default-limit 30
```

安全確認中は `--rc-default-limit` を 20 から 30 程度に制限する。いきなり 100 にしない。

6 PC から Raspberry Pi へ SSH/SCP するためのネットワーク設定

本節では, PC から Raspberry Pi を遠隔操作するための SSH/SCP 設定をまとめる。本構成では, Raspberry Pi は Tello と Wi-Fi で 1 対 1 接続しつつ, PC とはスイッチングハブ経由の有線 LAN で接続される。したがって, PC から Raspberry Pi へ SSH する際は, Tello 側の Wi-Fi アドレスではなく, Raspberry Pi の有線 LAN 側 IP アドレスを指定する。

6.1 二つのネットワークを分離する考え方

Raspberry Pi は次の二つのネットワークに同時に参加する。

```
eth0 または end0 : PC との制御・SSHSCP 用
wlan0             : Tello との SDK 通信用
```

たとえば Pi-1 では次のように設定する。

```
PC Ethernet : 192.168.50.10/24
Pi eth0     : 192.168.50.101/24
Pi wlan0    : 192.168.10.x/24
Tello       : 192.168.10.1
```

有線 LAN 側と Tello Wi-Fi 側を同じ 192.168.10.0/24 にしてはいけない。Tello は各機体で同じ 192.168.10.1 を使うため、有線 LAN 側は 192.168.50.0/24 などの別ネットワークに分ける。

6.2 PC 側 Ethernet の固定 IP 設定

Windows PC 側では、スイッチングハブに接続している Ethernet アダプタに固定 IP を設定する。

```
アドレス
IP          : 192.168.50.10サブネットマスク
            : 255.255.255.0デフォルトゲートウェイ
            : 空欄
DNS         : 空欄
```

スイッチングハブだけでは DHCP により IP アドレスは配布されないため、PC と Raspberry Pi の両方に固定 IP を設定する。PC 側の設定確認は PowerShell で次を実行する。

```
ipconfig
```

6.3 Raspberry Pi 側有線 LAN の固定 IP 設定

Raspberry Pi 側では、有線 LAN の接続名を確認する。Raspberry Pi の機種や OS により、インターフェース名は eth0 ではなく end0 になる場合がある。

```
nmcli con show
ip -br addr
```

有線接続名が Wired connection 1 の場合、Pi-1 では次のように設定する。

```
sudo nmcli con mod "Wired connection 1" \
  ipv4.method manual \
  ipv4.addresses 192.168.50.101/24 \
  ipv4.gateway "" \
  ipv4.dns "" \
  ipv6.method disabled

sudo nmcli con up "Wired connection 1"
```

Pi-2 では 192.168.50.102/24、Pi-3 では 192.168.50.103/24 のように、Pi ごとに異なる IP アドレスを割り当てる。設定後、次のような状態になっていることを確認する。

```
ip -br addr
```

期待例

```
eth0    UP    192.168.50.101/24
wlan0   UP    192.168.10.x/24
```

または, Raspberry Pi 5 などでは次のように表示されることがある.

```
end0    UP    192.168.50.101/24
wlan0   UP    192.168.10.x/24
```

6.4 Tello Wi-Fi 接続時の注意

Raspberry Pi の Wi-Fi 側は, 対応する Tello の SSID に接続する. このとき, Tello Wi-Fi をデフォルト経路として使わないようにするため, `ipv4.never-default yes` を設定しておくといよい.

```
sudo nmcli dev wifi rescan
nmcli dev wifi list

sudo nmcli con add type wifi ifname wlan0 con-name tello1 ssid TELLO-XXXXXX
sudo nmcli con mod tello1 ipv4.method auto ipv4.never-default yes ipv6.method disabled
sudo nmcli con up tello1
```

接続後, Raspberry Pi から Tello に到達できることを確認する.

```
ping 192.168.10.1
```

6.5 SSH サーバの有効化

PC から SSH 接続するには, Raspberry Pi 側で SSH サーバが起動している必要がある. Raspberry Pi 上で次を実行する.

```
sudo systemctl enable --now ssh
sudo systemctl status ssh
```

SSH サーバが入っていない場合は, 次でインストールする.

```
sudo apt update
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
```

22 番ポートで待ち受けているかは次で確認できる.

```
sudo ss -tlnp | grep :22
```

6.6 PC から SSH 接続する

PC の PowerShell から、有線 LAN 側 IP アドレスを指定して接続する。

```
ssh ユーザー名@192.168.50.101
```

たとえば、Raspberry Pi のユーザー名が inulab-rpi-01 の場合は次のようにする。

```
ssh inulab-rpi-01@192.168.50.101
```

ユーザー名が分からない場合、Raspberry Pi にログイン済みの端末で次を実行する。

```
whoami
```

表示された文字列をそのまま ssh や scp のユーザー名として使う。

6.7 SCP によるファイル転送

scp は SSH 経由でファイルを転送するコマンドである。PC から Raspberry Pi へファイルを送る場合は、PC 側の PowerShell で実行する。Raspberry Pi に SSH ログインした状態で Windows の C: ドライブを指定しても、Raspberry Pi から Windows のローカルファイルは見えない。

PC から Pi-1 へ tello_gateway.py を送る例を示す。

```
scp .\tello_pi_gateway_patch\raspberry_pi\tello_gateway.py \  
inulab-rpi-01@192.168.50.101:~/tello_gateway.py
```

絶対パスで指定する場合は次のようにする。

```
scp "C:\Users\inulab\Work\Tello-drone\tello_pi_gateway_patch\raspberry_pi\tello_gateway.  
py" \  
inulab-rpi-01@192.168.50.101:~/tello_gateway.py
```

転送後、SSH で Raspberry Pi に入り、ファイルが存在するか確認する。

```
ssh inulab-rpi-01@192.168.50.101  
ls -l ~/tello_gateway.py
```

Raspberry Pi から PC へログを回収する場合は、PC 側 PowerShell で次のように実行する。

```
scp inulab-rpi-01@192.168.50.101:/home/inulab-rpi-01/log.csv .\
```

6.8 SSH/SCP の典型的なエラーと対処

6.8.1 ping がタイムアウトする

ping 192.168.50.101 がタイムアウトする場合は、SSH 以前に有線 LAN 通信が成立していない。次を確認する。

- PC の Ethernet IP が 192.168.50.10/24 になっているか。

- Raspberry Pi の有線 LAN IP が 192.168.50.101/24 になっているか.
- PC と Pi が同じスイッチングハブに接続されているか.
- LAN ケーブルまたはハブのリンクランプが点灯しているか.
- Pi 側で `ip link` を実行したとき、有線インターフェースが `NO-CARRIER` になっていないか.

Pi 側の物理リンクは次で確認する.

```
ip link
```

`LOWER_UP` が表示されていれば物理リンクは成立している. `NO-CARRIER` の場合は、ケーブル、ハブ、ポート、電源を確認する.

6.8.2 Connection refused

`ssh: connect to host 192.168.50.101 port 22: Connection refused` と表示される場合、IP には到達しているが、Raspberry Pi 側で SSH サーバが起動していない可能性が高い. Pi 側で次を実行する.

```
sudo systemctl enable --now ssh
sudo systemctl status ssh
```

6.8.3 Could not resolve hostname c

Raspberry Pi に SSH ログインした状態で次のようなコマンドを実行すると、`C:/Users/...` の `C:` がリモートホスト指定と解釈され、`Could not resolve hostname c` になる.

```
scp "C:/Users/inulab/Work/.../tello_gateway.py" \
inulab-rpi-01@192.168.50.101:~/tello_gateway.py
```

PC 上のファイルを Pi に送る `scp` は、Raspberry Pi 上ではなく、Windows PowerShell 上で実行する.

6.8.4 Permission denied

`scp` または `ssh` で `Permission denied` になる場合は、ユーザー名またはパスワードが違う可能性が高い. 今回の例では、プロンプトが次のように表示されている場合、ユーザー名は `inulab-rpi-01` である.

```
inulab-rpi-01@raspberrypi:~ $
```

この場合、アンダースコアではなくハイフンを使う.

```
scp .\tello_pi_gateway_patch\raspberry_pi\tello_gateway.py \
inulab-rpi-01@192.168.50.101:~/tello_gateway.py
```

6.9 推奨する確認順序

作業時は次の順に確認すると、問題を切り分けやすい.

1. PC の Ethernet IP が 192.168.50.10/24 であることを確認する.
2. Raspberry Pi の有線 LAN IP が 192.168.50.101/24 であることを確認する.
3. Raspberry Pi の Wi-Fi が Tello に接続され, wlan0 に 192.168.10.x/24 が付いていることを確認する.
4. PC から ping 192.168.50.101 が通ることを確認する.
5. Raspberry Pi で SSH サーバが起動していることを確認する.
6. PC から ssh ユーザー名@192.168.50.101 でログインする.
7. PC の PowerShell から scp で必要なファイルを転送する.
8. Raspberry Pi 上で ls -l により転送されたファイルを確認する.

7 PC 側の準備

7.1 ファイル配置

既存プロジェクトの Python ディレクトリに, 追加ファイルをコピーする. 既存構成が次のような場合を想定する.

```
project_root/ └──
python/
├── train.py
└── tello_rl/
    ├── config.py
    ├── formation.py
    ├── observation.py
    └── ...
```

ここに追加実装を次のように配置する.

```
project_root/ └──
python/
├── run_real_baseline.py
└── tello_rl/
    ├── real/
    │   ├── __init__.py
    │   ├── pi_client.py
    │   ├── real_tracker.py
    │   ├── real_safety_shield.py
    │   └── real_tello_env.py
```

設定ファイルは, 任意の場所に置いてよいが, 以下では project_root/configs/ に置く例とする.

```
project_root/ └──
configs/
├── pc_real_config.example.json
├── tracker_udp_schema.example.json
└── pi_gateway_commands.txt
```

7.2 Python パスの確認

PC 側では, 既存プロジェクトの `python/` ディレクトリで実行する.

```
cd project_root/python
python -c "import tello_rl; print('ok')"
python -c "from tello_rl.real import PiSwarmClient; print('real ok')"
```

インポートに失敗する場合は, 実行ディレクトリまたは `PYTHONPATH` を確認する.

```
# 例: project_root/python を PYTHONPATH に追加
export PYTHONPATH=$(pwd):$PYTHONPATH
```

Windows PowerShell の場合は次のようにする.

```
$env:PYTHONPATH = (Get-Location).Path + ";" + $env:PYTHONPATH
```

8 PC 側設定ファイル

`pc_real_config.example.json` をコピーし, 実験用に編集する.

```
cp ../configs/pc_real_config.example.json ../configs/pc_real_config.local.json
```

主に編集する項目は次である.

表 3: PC 側設定ファイルの主な項目

項目	意味
N	Tello の台数. 初期実験では 1, 次に 2, 最後に 3 と段階的に増やす.
dt	制御周期. 初期値は 0.1 秒程度. 実機では 5–10 Hz から始める.
pi_gateways	各 Raspberry Pi の名前, IP, ポートを列挙する.
tracker	外部計測 UDP を受信するホストとポート.
area	飛行領域の x, y, z 範囲. 実験室の実寸に合わせる.
safety	機体間距離, 壁距離, 高度範囲, 入力上限, 計測タイムアウトなどの安全設定.
controller	PD ベースラインのゲインや速度上限.

Pi の IP アドレスを実機環境に合わせて修正する. 例を以下に示す.

```
"pi_gateways": [
  {"name": "tello_1", "host": "192.168.50.101", "port": 10000},
  {"name": "tello_2", "host": "192.168.50.102", "port": 10000},
```

```
{ "name": "tello_3", "host": "192.168.50.103", "port": 10000 }
]
```

9 外部計測 UDP の形式

PC 側の `real_tracker.py` は、外部計測系から UDP JSON を受け取る。送信する JSON の基本形式は次の通りである。

```
{
  "t": 12.345,
  "target": {
    "r": [0.0, 0.0],
    "v": [0.0, 0.0]
  },
  "drones": [
    { "id": 0, "r": [0.9, 0.0], "h": 1.0, "yaw": 3.14 },
    { "id": 1, "r": [-0.45, 0.78], "h": 1.0, "yaw": -1.05 },
    { "id": 2, "r": [-0.45, -0.78], "h": 1.0, "yaw": 1.05 }
  ]
}
```

ここで、`r` は水平面座標 $[x, y]$ 、`h` は高度、`yaw` はラジアン単位の `yaw` 角である。Unity 座標系や画像座標系から変換する場合は、PC 側へ送る前に実験座標系へ変換しておく。

10 実装後の確認手順

10.1 Step 0: 構文チェック

まず、PC 側と Raspberry Pi 側の Python ファイルに構文エラーがないか確認する。

```
# PC 側
cd project_root/python
python -m py_compile run_real_baseline.py
python -m py_compile tello_rl/real/pi_client.py
python -m py_compile tello_rl/real/real_tracker.py
python -m py_compile tello_rl/real/real_safety_shield.py
python -m py_compile tello_rl/real/real_tello_env.py

# Raspberry Pi 側
python3 -m py_compile ~/tello_gateway.py
```

10.2 Step 1: Pi 単体で Tello SDK 通信確認

各 Pi で Tello Wi-Fi に接続し、ゲートウェイを起動する。PC から接続する前に、Pi 上で Tello に `command` が送られていること、バッテリー問い合わせができることを確認する。

ゲートウェイが PC から query メッセージを受けられる場合、PC から次のようなメッセージを送る。

```
{"type": "query", "cmd": "battery?"}
```

応答が返れば、Pi-Tello 間の UDP 通信は成立している。

10.3 Step 2: PC から Pi への接続確認

PC から各 Raspberry Pi へ TCP 接続できるか確認する。

```
ping 192.168.50.101
ping 192.168.50.102
ping 192.168.50.103
```

次に、PC 側プログラムから各 Pi へ rc 0 0 0 0 相当のメッセージを送り、ゲートウェイ側ログに受信が出ることを確認する。この段階では、Tello を離陸させない。

10.4 Step 3: 外部計測のみの確認

Tello を飛ばさず、外部計測系が UDP JSON を PC へ送信しているか確認する。PC 側の `real_tracker.py` が受信できる形式になっていることを確認する。

チェックすべき点は次である。

- Tello の個体 ID と `drones` 配列の ID が一致している。
- 単位が m と rad になっている。
- 座標系の x, y 軸向きが、制御で使う座標系と一致している。
- yaw 角の正方向が、Tello の前方方向と一致している。
- UDP 更新周期が制御周期より十分速いか、少なくとも同程度である。

10.5 Step 4: PC 側ベースラインを dry run する

`--takeoff` を付けずに PC 側ベースラインを実行する。この段階では、制御計算、外部計測受信、Pi への接続だけを確認する。

```
cd project_root/python
python run_real_baseline.py \
  --config ../configs/pc_real_config.local.json \
  --duration 20
```

ログ上で、次を確認する。

- 各 Tello の計測値が更新されている。
- 目標スロットが飛行領域内にある。
- 安全シールドが異常な補正を連発していない。
- Pi への送信が成功している。

10.6 Step 5: 単機 takeoff/land 確認

最初は $N = 1$ に設定し、Tello 1 台だけで確認する。外部計測が正常に入っている状態で、PC 側から takeoff と land を行う。

```
python run_real_baseline.py \  
--config ../configs/pc_real_config.local.json \  
--duration 10 \  
--takeoff
```

この段階では、制御入力を小さくし、必要であれば目標位置を現在位置付近に置く。正常に離陸し、ホバリングし、着陸できることを確認する。

10.7 Step 6: 単機位置制御

次に、単機で目標位置への PD 制御を確認する。初期設定では、速度上限と RC 上限を低くする。

- rc_limit: 20–30
- 水平速度上限: 0.2–0.3 m/s
- 高度目標: 0.8–1.0 m
- 制御周期: 0.1–0.2 s

機体が目標位置と逆方向に動く場合は、座標系または yaw 角の符号が合っていない。左右、前後、yaw の符号を単機で再確認する。

10.8 Step 7: 複数機の固定位置制御

単機で安定した後、 $N = 2$ 、次に $N = 3$ へ増やす。この段階では対象物取り囲みではなく、各機体に十分離れた固定目標位置を与える。安全距離を大きめに設定し、機体間距離が保たれることを確認する。

10.9 Step 8: 静止対象物の正多角形取り囲み

$N = 3$ 、対象物静止、Case 1 の正多角形配置を用いる。取り囲み半径 R は 0.9 m 程度から始める。初期実験では、対象物を床上の固定マーカーとし、外部計測系が対象物位置 r_o を送信する。

評価するログ項目は次である。

- slot 誤差 $\|r_i - r_i^{\text{slot}}\|$
- 対象物からの半径誤差
- 機体間距離
- 壁距離
- 高度誤差
- safety shield の補正回数

- Pi との通信遅延

10.10 Step 9: 移動対象物追従

静止対象物で安定してから、対象物を低速で動かす。対象物速度は最初は 0.05–0.1 m/s 程度に制限する。急な折れ線軌道や円軌道は、静止対象物と等速直線追従が安定してから行う。

10.11 Step 10: 学習済み方策への置き換え

PD または NSB ベースラインで安全性を確認した後、学習済み方策を `RealTelloEnv` に接続する。このときも、安全シールドは無効化しない。最初は探索ノイズを切り、方策の平均 action のみを用いる。RC 上限も低い値から始める。

11 安全確認チェックリスト

実機飛行前に、以下を確認する。

1. 各 Raspberry Pi から対応する Tello にだけ接続されている。
2. PC から各 Raspberry Pi に ping が通る。
3. 外部計測系が全 Tello と対象物を認識している。
4. Tello ID と Pi ID と外部計測 ID の対応が一致している。
5. 座標系の x, y 軸, yaw 正方向, Tello 前方方向が一致している。
6. PC 側安全シールドが有効である。
7. Pi 側 watchdog が有効である。
8. PC 側プログラムを停止した場合に、Pi が `rc 0 0 0 0` または `land` を送る。
9. `emergency` を送る方法が確認されている。
10. バッテリ残量が十分である。
11. 飛行領域に人や壊れやすい物がない。
12. 初回は必ず低い RC 上限で実行する。

12 トラブルシューティング

12.1 PC から Pi に接続できない

次を確認する。

- Pi の有線 LAN 側 IP が設定通りか。
- PC と Pi が同じサブネットにいるか。
- `tello_gateway.py` が起動しているか。
- ファイアウォールが TCP ポート 10000 をブロックしていないか。

12.2 Pi から Tello に接続できない

次を確認する.

- Pi の wlan0 が正しい Tello SSID に接続されているか.
- ping 192.168.10.1 が通るか.
- 他の端末が同じ Tello Wi-Fi に接続していないか.
- Tello のバッテリーが十分か.

12.3 Tello が逆方向に動く

座標系または符号規約の問題である可能性が高い. 単機で次を確認する.

- rc 0 20 0 0 で機体が前方へ動くか.
- rc 20 0 0 0 で左右のどちらへ動くか.
- yaw 角が増える方向と画像処理で推定した yaw の正方向が一致しているか.
- 世界座標速度から機体座標速度への変換式が正しいか.

12.4 安全シールドが常に停止を返す

次を確認する.

- 外部計測の座標単位が cm ではなく m になっているか.
- 飛行領域設定 area が実験室座標と一致しているか.
- 高度 h の値が正しく入っているか.
- Tello ID と計測 ID が入れ替わっていないか.
- tracker timeout が短すぎないか.

13 実験ログの保存項目

実機実験では, 既存の目的コスト・制約コスト・取り囲み誤差に加えて, 通信系と安全系のログを保存する. 最低限保存すべき項目は次である.

- 制御ステップ k と時刻 t .
- 各 Tello の位置 r_i , 高度 h_i , yaw 角 ψ_i .
- 対象物位置 r_o , 対象物速度 v_o .
- 目標スロット r_i^{slot} .
- raw action $\tilde{a}_i$.
- executed action a_i .
- 実際に送信した RC 値.

- 安全シールドが action を補正した理由.
- 各 Pi への送信成功・失敗.
- PC-Pi 間通信遅延.
- Tello のバッテリー残量.
- 外部計測の最終更新時刻.
- watchdog 発火の有無.

14 既存学習コードとの接続方針

追加実装のうち, `real_tello_env.py` は, 既存の `MockEnv` や `UnityEnv` と同じ形式の API を提供することを目的としている. すなわち, 外部からは次のように扱う.

```
obs = env.reset()
obs, obj, con, exe, done, info = env.step(raw_actions)
```

ただし, 実機環境では, `step` の中で数値モデルを前進させるのではなく, 次を行う.

1. raw action を安全シールドに通す.
2. executed action を各 Raspberry Pi へ送る.
3. 次の制御周期まで待つ.
4. 外部計測から実状態を取得する.
5. 観測, 目的コスト, 制約コストを計算して返す.

このため, 学習済み方策の実機評価は可能であるが, 実機上でオンライン学習を行う場合は, 安全性, 試行回数, バッテリー, 計測欠落時の扱いを別途慎重に設計する必要がある. 初期段階では, シミュレーションまたは Unity で学習した方策を, 実機では評価のみ行う構成が望ましい.

15 まとめ

本追加実装では, 通常 Tello の Wi-Fi 制約を回避するために, Tello 1 台につき Raspberry Pi 1 台を割り当てるゲートウェイ構成を採用する. PC は全体の状態推定, 安全評価, 制御計算を担当し, Raspberry Pi は対応する Tello への SDK コマンド送信を担当する. この分担により, 既存の command-level action 表現を保ったまま, 通常 Tello 複数台の実機実験へ移行できる.

実機投入は, 必ず単機通信確認, 外部計測確認, 単機位置制御, 複数機固定位置制御, 静止対象物取り囲み, 移動対象物追従, 学習済み方策評価の順に進める. 特に, 外部計測と安全シールドが安定するまでは, RL 方策を直接実機へ接続しない.